Project 2 on Neural networks

Tim Herrmann Holmen

FYS-STK4155
Department of Physics

This project ...

CONTENTS

I. Introduction 2

II. Theory 2
A. Cost functions and their gradients 2
1. MSE 2
2. Binary cross entropy: log loss 2
3. Multiclass cross entropy cost function 2
B. Activation functions and their gradients 3
1. Sigmoid logit function 3
2. RELU function 3
3. leaky RELU function 3

III. Methods 3
A. NeuralNetwork 3
1. Feed-forward pass 3
2. Backpropagation 3
3. Analysis 3
B. Comparison with OLS 3
C. Activation, depth and width 3
1. Ridge 3
2. Lasso 3
D. Classification 3
E. Measure of quality 3
1. Confusion matrix 3
F. ROC and AUC 3

IV. Results 3

V. Discussion 3

VI. Conclusion 3

A. Use of AI/LLM Tools 3
1. Tools used 3
2. Text contributions 3
3. Code contributions 4
a. LLM Declaration this section: 4

References 4

I. INTRODUCTION

II. THEORY

A. Cost functions and their gradients

The loss function is the most important function for training, since training a model means to minimize the cost function of the model. To minimize the cost function, we use backpropagation to find the gradient of the cost, with regards to and gradient descent.

1. MSE

When we have our calculated output value $\tilde{y}$ and the true output value y, with n data points, the mean-squared error and its derivative is

$$MSE(y,\tilde{y}) = \frac{1}{n}\sum_{i=0}^{n-1}(y-\tilde{y})^2 \quad \frac{1}{n}||y - X\Theta||_2^2 \tag{1}$$

a. L2 norm which corresponds to the Ridge cost function.

$$MSE + \lambda||\Theta||_2^2 \quad \Theta(W,b) \tag{2}$$

(Hjorth-Jensen, 2016, p. 86)

b. L1 norm which corresponds to the Lasso cost function.

$$MSE + \lambda||\Theta|| \quad \Theta(W,b) \tag{3}$$

(Hjorth-Jensen, 2016, p. 90-91)

2. Binary cross entropy: log loss

With two classes, we assume a binary output $y \in \{0,1\}$, and a predicted probability $p \in [0,1]$ which is the output of the sigmoid function of z_i. Where $z_i = x_i^T\Theta$ is the linear combination of the input features and the model parameters Θ. For a single example, data point i, the binary cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log[1-p_i]) \tag{4}$$

$$p_i = \sigma(z_i) = \frac{1}{1+\exp(-z_i)} \tag{5}$$

$$\tag{6}$$

The cost function is derived from the maximum likelihood estimation of the Bernoulli distribution, the negative log likelihood of the Bernoulli distribution. This is known as the cross entropy, and it is the loss function used to train binary classification models. (Hjorth-Jensen, 2016, p. 151) With the linear function $z_i = x_i^T\Theta$, the gradient of the cost function with respect to the model parameters Θ is given by

$$\frac{dC(\theta)}{d\theta} = -X^T(y-p) \tag{7}$$

For N data points, the mean binary cross entropy is defines as

$$\frac{1}{N}\sum_{i=1}^{N} C(\theta)_i$$

And the term $1/N$ follows through te calculations to give the mean binary cross entropy gradient, which is given by

$$\nabla_\Theta C(\Theta) = -\frac{1}{n}X^T(y-p) \tag{8}$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE. (Hjorth-Jensen, 2016, p. (151-154))

3. Multiclass cross entropy cost function

For k klasses, one uses the soft max function to calculate the probability of each class. Again we have the linear function $z_k = X^T\Theta_k$ for each class, over all features and data points, the softmax functions is defined as

$$p(C=k \mid x) = \frac{\exp(z_k)}{\sum_{l=0}^{K-1}\exp(z_l)} k = 0,\ldots,K-1 \tag{9}$$

The nominator is for class k, and the denominator is the sum of all classes. The multiclass cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log p(C=k|x_i) \tag{10}$$

Which is summed over all data points and classes. The gradient of the multiclass cross entropy cost function is given by

$$\frac{dC}{d\Theta_k} = -X^T(y_k - p_k) \tag{11}$$

(Hjorth-Jensen, 2016, p. 156) The cross entropy L1 and L2 norm is supplemented like the ordinary MSE. Stability: The softmax function can be numerically unstable for large values of z_k, for a large value the e exponential function can overflow. Since the softmax is invariant udner subtracting a constant frome every score, one subtracts the largest before exponentiating:

$$p_k = \frac{\exp(z_k - z_{max})}{\sum_l \exp(z_l - z_{max})}$$

, $z_k = x^T\Theta_k$. (Hjorth-Jensen, 2016, p. 156)

B. Activation functions and their gradients

1. Sigmoid logit function

$$\sigma(z) = \frac{1}{1+\exp(-z)} \tag{12}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{13}$$

(Hjorth-Jensen, 2016, p. 214-242)

2. RELU function

$$ReLU(z) = \max(0, z) \tag{14}$$

$$ReLU'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases} \tag{15}$$

(Hjorth-Jensen, 2016, p. 214-242)

3. leaky RELU function

$$leakyReLU(z) = \begin{cases} 0.01z & z < 0 \\ z & z > 0 \end{cases} \tag{16}$$

$$leakyReLU'(z) = \begin{cases} 0.01 & z < 0 \\ 1 & z > 0 \end{cases} \tag{17}$$

(Hjorth-Jensen, 2016, p. 214-242)

III. METHODS

A. NeuralNetwork

1. Feed-forward pass

2. Backpropagation

3. Analysis

B. Comparison with OLS

C. Activation, depth and width

1. Ridge

2. Lasso

D. Classification

E. Measure of quality

1. Confusion matrix

F. ROC and AUC

IV. RESULTS

V. DISCUSSION

VI. CONCLUSION

Appendix A: Use of AI/LLM Tools

1. Tools used

List every LLM tool used for the project, including its model or version, how it was accessed, and the approximate dates of use.

- *[Tool/version; access method; date used]*
- *GPT 6-Luna (codex), used 22.10–06.11.2026*
- *[GPT 6-Luna/version; GitHub Copilot (coding assistant session);* 22.10 – 06.11.2026*]*

2. Text contributions

For each report section, enter the highest applicable contribution level and briefly describe the assistance. Use level 0 for no assistance, level 1 for language-only edits, level 2 for editorial changes, and level 3 for substantial drafting. Replace the placeholders with an accurate record.

Abstract:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Introduction:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Theory:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Methods:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Results:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Discussion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Conclusion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Level 3 prompt and revision note (delete if not applicable): [Summarise the prompt or task, how the generated text was changed, and how it was checked.]

3. Code contributions

List each code file or notebook included in the submission. Use level 0 for no assistance, level 1 for debugging, level 2 for a snippet, level 3 for a skeleton, and level 4 for substantial generation. Include what was tested, adapted, or verified.

`[Path / filename]::` Level *[0–4]*; *[Describe assistance and your testing/adaptations]*

Duplicate the code entry above for every submitted code file or notebook.
For every LLM-assisted function at level 2 or above, add an `LLM-assisted` note to its docstring. Add an inline note for assisted code blocks shorter than a function, and a Markdown note above assisted notebook cells.

a. LLM Declaration this section:

The skeleton of this section was created using GitHub Copilot. (modelversion unknown; coding-assistant session, November 2, 2026).

REFERENCES

M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).